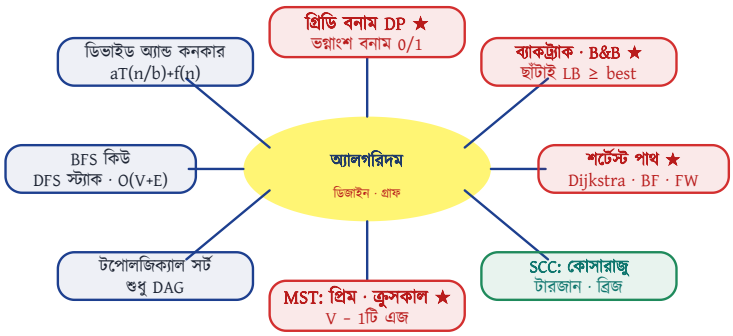
অ্যালগরিদম ডিজাইন পদ্ধতি ও গ্রাফ অ্যালগরিদম

ডিভাইড অ্যান্ড কনকার · গ্রিডি · DP · ব্যাকট্র্যাকিং · ব্রাঞ্চ অ্যান্ড বাউন্ড · BFS/DFS · টপোলজিক্যাল সর্ট · SCC · MST · শর্টেস্ট পাথ: ১১ পাতায় পরীক্ষার প্রস্তুতি



↑ এক নজরে: লাল = হাই-ইল্ড, সবুজ = DFS-পরিবার

সিলেবাসের যে অংশ

Data Structures and Algorithms অংশের লাইন: "...**Methods for the design of efficient algorithms: divide and conquer, greedy method, dynamic programming, back tracking, branch and bound; Basic search and traversal techniques; Topological sorting; Connected components, spanning trees, shortest paths;** Flow algorithms; ..." এই সেট ডিজাইন পদ্ধতি থেকে শর্টেস্ট পাথ পর্যন্ত সবটা নেয়। **মাস্টার থিওরেম** ও সর্টিং-এর খুঁটিনাটি রিকার্শন ও সর্টিং লেসনে, ডায়াক্সট্রা/প্রিমের পেছনের **হিপ** হিপ লেসনে, **ফ্লো** ফ্লো ও অ্যাপ্রক্সিমেশন লেসনে, আর **NP-কমপ্লিটনেস** শেষ লেসনে; গ্রাফ তত্ত্বের মৌলিক বিষয় অধ্যায় ১৩-তে।

হাই-ইল্ড তালিকা

① মার্জ সর্ট $2T(n/2)+n = \Theta(n \log n)$
② স্ট্র্যাসেন (Strassen) $O(n^{\log_2 7}) \approx O(n^{2.81})$
③ v/w অনুপাতে গ্রিডি: শুধু **ভগ্নাংশ** ন্যাপস্যাক
④ DP = ওভারল্যাপিং + অপটিমাল সাবস্ট্রাকচার
⑤ ফ্লয়েড-ওয়ারশাল ও বেলম্যান-ফোর্ড DP
⑥ ব্যাকট্র্যাকিং = স্টেট-স্পেসের DFS
⑦ B&B (min): LB ≥ best হলে ছাঁটাই
⑧ BFS/DFS: লিস্টে $O(V+E)$, ম্যাট্রিক্সে $O(V^2)$
⑨ টপোলজিক্যাল ক্রম ⇔ DAG
⑩ MST-তে ঠিক V − 1টি এজ
⑪ ডায়াক্সট্রা: ঋণাত্মক এজ চলবে না
⑫ বেলম্যান-ফোর্ড $O(VE)$, ঋণাত্মক চক্র ধরে

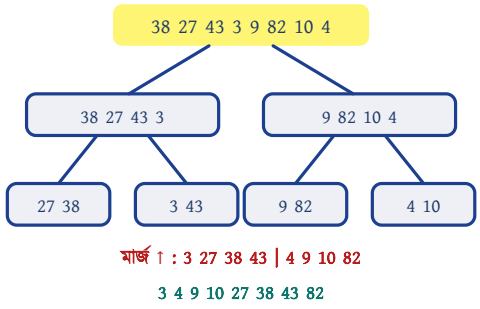
সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি** = মনে রাখার কৌশল · **লাল** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

সূচিপত্র

✓ ডিভাইড অ্যান্ড কনকার ... পৃ. ২
✓ গ্রিডি পদ্ধতি ... পৃ. ৩
✓ DP: ফিবোনাচ্চি, LCS ... পৃ. ৪
✓ DP: ন্যাপস্যাক, ম্যাট্রিক্স চেইন ... পৃ. ৫
✓ ব্যাকট্র্যাকিং, ব্রাঞ্চ অ্যান্ড বাউন্ড ... পৃ. ৬
✓ BFS, DFS, টপোলজিক্যাল সর্ট ... পৃ. ৭
✓ কম্পোনেন্ট, SCC, MST ... পৃ. ৮
✓ শর্টেস্ট পাথ ... পৃ. ৯
✓ পরীক্ষার আগের রাতের পাতা ... পৃ. ১০
✓ অনুশীলনী MCQ + লিখিত ... পৃ. ১১

১ ডিভাইড অ্যান্ড কনকার: ভাগ, সমাধান, জোড়া

সংজ্ঞা: **ডিভাইড অ্যান্ড কনকার** (Divide and Conquer) তিন ধাপে সমস্যা সমাধান করে: (১) **ভাগ (Divide)**: সমস্যাকে একই ধরনের a-টি ছোট উপসমস্যায় ভাঙা, (২) **জয় (Conquer)**: প্রতিটি রিকার্সিভভাবে সমাধান (খুব ছোট হলে সরাসরি), (৩) **জোড়া (Combine)**: উপ-উত্তরগুলো মিলিয়ে মূল উত্তর। উপসমস্যাগুলো **স্বাধীন** (ওভারল্যাপ করে না) — এটাই ডায়নামিক প্রোগ্রামিং থেকে পার্থক্য। খরচের রিকারেন্স **T(n) = aT(n/b) + f(n)**: n/b আকারের a-টি উপসমস্যা, f(n) = ভাগ + জোড়ার কাজ।



মার্জ সর্ট: $\log_2 n$ স্তর × প্রতি স্তরে n কাজ = n log n

অ্যালগরিদম	রিকারেন্স	ফল
বাইনারি সার্চ	T(n/2) + 1	Θ(log n)
মার্জ সর্ট	2T(n/2) + n	**Θ(n log n)**
কুইক সর্ট সেরা/গড়	2T(n/2) + n	Θ(n log n)
কুইক সর্ট খারাপতম	T(n-1) + n	$\Theta(n^2)$
ম্যাক্স-মিন (n টি)	2T(n/2) + 2	3n/2 − 2 তুলনা
কারাৎসুবা	3T(n/2) + n	$n^{1.585}$
স্ট্র্যাসেন	7T(n/2) + n^2	$\mathbf{n^{2.81}}$
ক্লোজেস্ট পেয়ার	2T(n/2) + n	Θ(n log n)

সমাধানকৃত উদাহরণ ✍ স্ট্র্যাসেন: সাধারণ ব্লক-গুণে n×n ম্যাট্রিক্সকে চারটি n/2 ব্লকে ভেঙে **৮টি** ব্লক-গুণফল লাগে: $T(n) = 8T(n/2) + n^2 = \Theta(n^3)$। স্ট্র্যাসেন কাজটি সাজিয়ে **৭টি** গুণফলে (আর ১৮টি যোগে) নামান: $T(n) = 7T(n/2) + \Theta(n^2)$। যেহেতু $\log_2 7 \approx 2.807 > 2$, পাতাগুলোর খরচই প্রধান: $\mathbf{\Theta(n^{\log_2 7}) \approx O(n^{2.81})}$।

দ্রুত সমাধান: f(n)-কে $n^{\log_b a}$-এর সাথে তুলনা করুন। পাতা ভারী → $\Theta(n^{\log_b a})$; সমান → $\Theta(n^{\log_b a} \cdot \log n)$; মূল ভারী → Θ(f(n))। নিয়মিততা শর্তসহ পূর্ণ মাস্টার থিওরেম: রিকার্শন ও সর্টিং লেসন।

পরীক্ষার ফাঁদ: (i) কুইক সর্টের **জোড়া** ধাপ ফাঁকা, তবু এটি D&C — কাজ হয় **পার্টিশনে**। মার্জ সর্ট উল্টো: ভাগ সহজ, জোড়াতেই আসল কাজ। (ii) বাইনারি সার্চ **একটি** অর্ধেকই রাখে (a = 1), তাই একে কখনো "ডিক্রিজ অ্যান্ড কনকার" বলে।

মনে রাখার কৌশল: "**ভাগ-জয়-জোড়া (D-C-C)**।" স্ট্র্যাসেন: "**আট নয়, সাত**" → $\log_2 7 \approx 2.81$। কারাৎসুবা: "**চার নয়, তিন**" → $\log_2 3 \approx 1.585$।

কখন D&C খাটে: উপসমস্যা একই ধরনের, স্বাধীন ও প্রায় সমান আকারের হলে। একই উপসমস্যা বারবার ফিরে এলে (সাধারণ রিকার্সিভ ফিবোনাচ্চি) DP-তে যান।

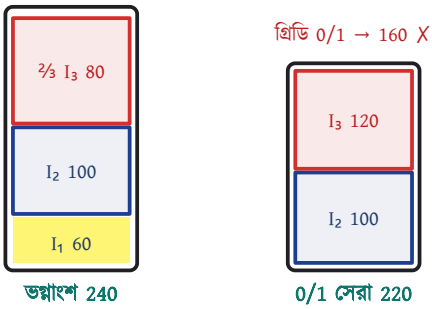
সমাধানকৃত উদাহরণ ✍ বাইনারি সার্চ: A = [3, 9, 14, 21, 27, 35, 42], কী 27। mid = A[3] = 21 < 27 → ডান অর্ধেক [27, 35, 42]; mid = 35 > 27 → বাম অর্ধেক [27]; পাওয়া গেল। **৩টি তুলনা** = $\lfloor \log_2 7 \rfloor + 1$, n = 7-এর খারাপতম ক্ষেত্র। দরকার **সাজানো** অ্যারে ও র‍্যান্ডম অ্যাক্সেস।

```
MergeSort(A, l, r):
  if l < r:
    m = (l + r) / 2
    MergeSort(A, l, m)       // conquer
    MergeSort(A, m + 1, r)   // conquer
    Merge(A, l, m, r)        // combine, O(n
```

২ গ্রিডি পদ্ধতি: এখনই সেরাটা নাও, আর ফিরো না

সংজ্ঞা: **গ্রিডি অ্যালগরিদম (Greedy Algorithm)** ধাপে ধাপে সমাধান গড়ে; প্রতি ধাপে **এই মুহূর্তে** যেটি সেরা মনে হয় (লোকাল অপটিমাম) সেটি নেয় এবং কখনো ফিরে দেখে না। গ্লোবাল অপটিমাম তখনই মেলে যখন সমস্যায় দুটি ধর্মই থাকে: **গ্রিডি-চয়েস ধর্ম (greedy-choice property)** (কোনো একটি অপটিমাল সমাধান গ্রিডি পছন্দ দিয়েই শুরু হয়) এবং **অপটিমাল সাবস্ট্রাকচার (optimal substructure)** (ওই পছন্দের পর বাকিটা ছোট সমস্যার অপটিমাল সমাধান)। নইলে উত্তর দ্রুত আসে, কিন্তু ভুলও হতে পারে।

সমাধানকৃত উদাহরণ ✍️ ভগ্নাংশ ন্যাপস্যাক (Fractional Knapsack), W = 50:
আইটেম (মান, ওজন): I_1 (60, 10), I_2 (100, 20), I_3 (120, 30)। অনুপাত v/w = **6, 5, 4**। অনুপাত অনুযায়ী সাজিয়ে ভরুন: পুরো I_1 (w 10, v 60), পুরো I_2 (w 30, v 160), তারপর I_3-এর 20/30 অংশ → 120 × 2/3 = 80। **মোট = 240**, অপটিমাল। খরচ: সাজাতে O(n log n)।



একই আইটেম, দুই সমস্যা, দুই উত্তর

পরীক্ষার ফাঁদ: এখানে গ্রিডি ব্যর্থ: (i) **0/1 ন্যাপস্যাক** (আইটেম ভাঙা যায় না): অনুপাত ধরে গ্রিডি নেয় $I_1 + I_2 = 160$, অথচ অপটিমাম $I_2 + I_3 = 220$। এখানে DP লাগবে। (ii) **কয়েন চেঞ্জ**, কয়েন {1, 3, 4}, পরিমাণ 6: গ্রিডি দেয় 4 + 1 + 1 (৩টি কয়েন), অপটিমাম 3 + 3 (২টি কয়েন)। গ্রিডি শুধু 1, 2, 5, 10-এর মতো "ক্যানোনিকাল" ব্যবস্থায় অপটিমাল।

গ্রিডি অ্যালগরিদম	গ্রিডি নিয়ম	সময়
ভগ্নাংশ ন্যাপস্যাক	সর্বোচ্চ মান/ওজন আগে	O(n log n)
অ্যাক্টিভিটি সিলেকশন	যেটি আগে **শেষ** হয় সেটি আগে, সংঘাত বাদ	O(n log n)
হাফম্যান কোডিং	সবচেয়ে কম ফ্রিকোয়েন্সির দুটি নোড জোড়া (min-heap)	O(n log n)
ডেডলাইনসহ জব সিকোয়েন্সিং	বেশি লাভ আগে, ডেডলাইনের মধ্যে শেষ খালি স্লট	$O(n^2)$
প্রিম (Prim) / ক্রুসকাল (Kruskal)	সবচেয়ে সস্তা নিরাপদ এজ	পৃ. ৮
ডায়াক্সট্রা (Dijkstra)	সবচেয়ে কাছের অমীমাংসিত ভার্টেক্স স্থির	পৃ. ৯

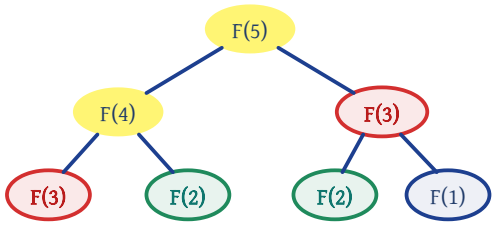
সমাধানকৃত উদাহরণ ✍️ অ্যাক্টিভিটি সিলেকশন: (শুরু, শেষ): A(1,4) B(3,5) C(0,6) D(5,7) E(3,9) F(5,9) G(6,10) H(8,11) I(8,12) J(2,14) K(12,16)। শেষ-সময়ে সাজিয়ে: A নিন (শেষ 4) → শুরু ≥ 4 এমন প্রথমটি D (শেষ 7) → H (শেষ 11) → K। **৪টি কাজ** {A, D, H, K}। *শুরুর* সময় বা *সবচেয়ে ছোট* সময়কাল ধরে সাজালে সবসময় কাজ করে না।

	গ্রিডি	DP
পছন্দ	একটি, আর ফেরে না	সব, সেরাটা রাখে
লাগে	গ্রিডি-চয়েস + অপ. সাবস্ট্র.	ওভারল্যাপ + অপ. সাবস্ট্র.
গতি	দ্রুততর	ধীর, টেবিল লাগে
ন্যাপস্যাক	ভগ্নাংশ ✓	0/1 ✓

মনে রাখার কৌশল: **"গ্রিডি খপ করে ধরে, DP ভেবে দেখে।"** ভাগ করা যায় এমন জিনিস (সোনার গুঁড়া) → গ্রিডি; ভাগ করা যায় না (সোনার বার) → DP।

৩ ডায়নামিক প্রোগ্রামিং: প্রতিটি উপসমস্যা একবারই

সংজ্ঞা: **ডায়নামিক প্রোগ্রামিং** (Dynamic Programming, DP) সমস্যাকে এমন উপসমস্যায় ভাঙে যেগুলো **ওভারল্যাপ** করে; প্রতিটি একবারই সমাধান করে উত্তর টেবিলে রাখে। দুটি ধর্ম লাগে: **ওভারল্যাপিং উপসমস্যা** (overlapping sub-problems) (একই উপসমস্যা বারবার আসে) এবং **অপটিমাল সাবস্ট্রাকচার** (অপটিমাল সমাধান অপটিমাল উপ-সমাধান দিয়ে গঠিত)। ধাপ: উপসমস্যা ঠিক করা → রিকারেন্স লেখা → বেস কেস → নিরাপদ ক্রমে টেবিল ভরা → পেছনে ফিরে উত্তর বের করা।



F(3) দুবার, F(2) তিনবার ...
সাধারণ কল বাড়ে $\varphi^n \approx 1.618^n$ হারে

একই রং = একই উপসমস্যা আবার সমাধান

	মেমোইজেশন	ট্যাবুলেশন
দিক	**টপ-ডাউন:** রিকার্শন + ক্যাশ	**বটম-আপ:** লুপ
সমাধান করে	শুধু দরকারি উপসমস্যা	টেবিলের সব ঘর
বাড়তি খরচ	কল স্ট্যাক	নেই; মেমরি কমানো যায়

ফিবোনাচ্চি টেবিল: F[i] = F[i−1] + F[i−2], F[0] = 0, F[1] = 1।

i	0	1	2	3	4	5	6
F	0	1	1	2	3	5	8

সাধারণ রিকার্শন: এক্সপোনেনশিয়াল; DP: **O(n) সময়**, দুটি ভেরিয়েবলে O(1) মেমরি।

লংগেস্ট কমন সাবসিকোয়েন্স (LCS)

রিকারেন্স: c[i][j] = 0 যদি i = 0 বা j = 0; **c[i−1][j−1] + 1** যদি $x_i = y_j$; নইলে **max(c[i−1][j], c[i][j−1])**। সময় ও মেমরি **O(mn)**। সাবসিকোয়েন্সে ক্রম ঠিক থাকে কিন্তু পাশাপাশি হতে হয় না (সাবস্ট্রিংকে হতে হয়)।

মনে রাখার কৌশল: "**মিললে → কোনাকুনি + 1; না মিললে → উপর ও বামের max।**" নিচের-ডান ঘর থেকে পেছনে ফিরুন: মিলের ঘরে কোনাকুনি চালগুলোই LCS।

সমাধানকৃত উদাহরণ ✍️ X = ABCB, Y = BDCAB:

	Ø	B	D	C	A	B
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	1	2	2	3

LCS-এর দৈর্ঘ্য **৩** = "BCB" (দুটিতেই B...C...B)।

পরীক্ষার ফাঁদ: (i) সব সমস্যায় DP মানে "টেবিলসহ রিকার্শন" নয়: **ওভারল্যাপিং** উপসমস্যা না থাকলে (মার্জ সর্ট) টেবিলে কিছুই বাঁচে না — সেটি সাধারণ D&C। (ii) m ও n দৈর্ঘ্যের LCS-এর খরচ O(mn), O(m + n) নয়। (iii) মেমোইজেশন ও ট্যাবুলেশনের অ্যাসিম্পটোটিক সময় **একই**; পার্থক্য শুধু ক্রম ও বাড়তি খরচে।

8 DP-র ক্লাসিক: ন্যাপস্যাক ও ম্যাট্রিক্স চেইন

0/1 ন্যাপস্যাক রিকারেন্স: K[i][w] = আইটেম 1..i দিয়ে ধারণক্ষমতা w-তে সর্বোচ্চ মান।
K[i][w] = K[i-1][w] যদি $w_i > w$; নইলে **max(K[i-1][w], v_i + K[i-1][w − w_i])**।
সময় **O(nW) — সিউডো-পলিনোমিয়াল** (pseudo-polynomial) (W-এর *মানে* পলিনোমিয়াল, এর বিট-দৈর্ঘ্যে এক্সপোনেনশিয়াল)।

পরীক্ষার ফাঁদ: ১-D সংস্করণ: একটি অ্যারে দিয়ে K[w] = max(K[w], v_i + K[w − w_i]) তখনই সঠিক যখন w **W থেকে w_i পর্যন্ত নিচের দিকে** চলে। w উপরের দিকে চালালে একই পাসে আইটেম i আবার ব্যবহৃত হয় — তখন সমাধান হয় **আনবাউন্ডেড** ন্যাপস্যাকের (অসীম কপি)।

সমাধানকৃত উদাহরণ ✍ W = 5: (w, v): 1 (2, 3) · 2 (3, 4) · 3 (4, 5) · 4 (5, 6)

i \ w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	**7**

K[2][5] = max(3, 4 + K[1][2]) = 7। উত্তর **7** = আইটেম **১** + **২** (ওজন 5)।

ম্যাট্রিক্স-চেইন গুণন (MCM)

সমস্যা: ম্যাট্রিক্স A_i-এর মাপ $p_{i-1} \times p_i$। $A_1A_2...A_n$-এর এমন বন্ধনী-বিন্যাস বাছতে হবে যাতে স্কেলার গুণ সবচেয়ে কম হয় (p×q গুণ q×r-এর খরচ pqr)। ক্রম বদলালে খরচ বদলায়, ফল নয়। রিকারেন্স: **m[i][j] = min (i ≤ k < j) m[i][k] + m[k+1][j] + $p_{i-1}p_kp_j$**, m[i][i] = 0। সময় **$O(n^3)$**, মেমরি $O(n^2)$। বন্ধনী-বিন্যাসের সংখ্যা কাতালান সংখ্যা C_{n-1}, তাই ব্রুট ফোর্স এক্সপোনেনশিয়াল।

সমাধানকৃত উদাহরণ ✍ ৩টি ম্যাট্রিক্স: A_1 10×30, A_2 30×5, A_3 5×60।
$(A_1A_2)A_3$ = 10·30·5 + 10·5·60 = 1500 + 3000 = **4500**
$A_1(A_2A_3)$ = 30·5·60 + 10·30·60 = 9000 + 18000 = 27000
একই গুণফল, ভুল ক্রমে **৬ গুণ কাজ**।

সমাধানকৃত উদাহরণ ✍ p = (5, 4, 6, 2, 7):

m	1	2	3	4
1	0	120	88	**158**
2		0	48	104
3			0	84

সেরা: **$((A_1(A_2A_3))A_4)$** = 48 + 40 + 70 = 158।

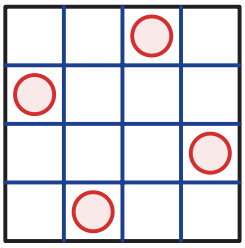
DP সমস্যা	উপসমস্যা	সময়
ফ্লয়েড-ওয়ারশাল (Floyd-Warshall)	$d_k[i][j]$: ভেতরে শুধু 1..k ভার্টেক্স ব্যবহার করা পাথ	$O(V^3)$
বেলম্যান-ফোর্ড (Bellman-Ford)	≤ k টি এজ ব্যবহার করে শর্টেস্ট পাথ	O(VE)
এডিট ডিস্ট্যান্স	দুই স্ট্রিংয়ের প্রিফিক্স	O(mn)
লংগেস্ট ইনক্রিজিং সাবসিকোয়েন্স	i-তে শেষ হওয়া LIS	$O(n^2)$ / O(n log n)
TSP (হেল্ড-কার্প)	(দেখা শহরের সেট, শেষ শহর)	$O(n^2 \cdot 2^n)$

মনে রাখার কৌশল: "**ফ্লয়েড প্রতিটি k-এর ভেতর দিয়ে ভাসে**" — বাইরের লুপটি মধ্যবর্তী ভার্টেক্স k, তাই ফ্লয়েড-ওয়ারশাল DP, গ্রিডি নয়।

আইটি সংযোগ: diff, Git ও DNA অ্যালাইনমেন্ট চলে LCS / এডিট ডিস্ট্যান্সে; SQL অপটিমাইজার জয়েনের ক্রম বাছে যেভাবে MCM বন্ধনী বাছে।

৫ ব্যাকট্র্যাকিং: স্টেট-স্পেসের DFS

সংজ্ঞা: **ব্যাকট্র্যাকিং** (Backtracking) একটি **স্টেট-স্পেস ট্রি** (state-space tree) ধরে একটি করে অংশ যোগ করে সমাধান গড়ে, ট্রিটি ঘোরে **ডেপথ-ফার্স্ট** ক্রমে। প্রতিটি নোডে একটি **বাউন্ডিং (সম্ভাব্যতা) ফাংশন** আংশিক সমাধান যাচাই করে; সেখান থেকে বৈধ উত্তরে পৌঁছানো অসম্ভব হলে পুরো সাবট্রি ছেঁটে আগের পছন্দে ফিরে যায়। ব্যবহার: N-কুইন, সাম অব সাবসেটস, গ্রাফ m-কালারিং, হ্যামিলটোনিয়ান সাইকেল, সুডোকু। খারাপতম ক্ষেত্রে এখনো এক্সপোনেনশিয়াল।



সারি ১ → কলাম ৩
সারি ২ → কলাম ১
সারি ৩ → কলাম ৪
সারি ৪ → কলাম ২

(3, 1, 4, 2) — আয়না (2, 4, 1, 3)

৪-কুইনের ঠিক ২টি সমাধান

সমাধানকৃত উদাহরণ ✍️ ৪-কুইন: প্রতি সারিতে একটি কুইন। (r_1, c_1) ও (r_2, c_2) সংঘাত করে যদি $c_1 = c_2$ বা $|r_1 - r_2| = |c_1 - c_2|$ (কর্ণ)। সারি ১ কলাম ১ → সারি ২ কলাম ৩ → সারি ৩-এ নিরাপদ কলাম নেই → সারি ২ কলাম ৪-এ ফেরা → সারি ৩ কলাম ২ → সারি ৪ অচল → সারি ১ কলাম ২-এ ফেরা → (2, 4, 1, 3) ✓। ৮-কুইন: **৯২টি সমাধান** (১২টি স্বতন্ত্র)।

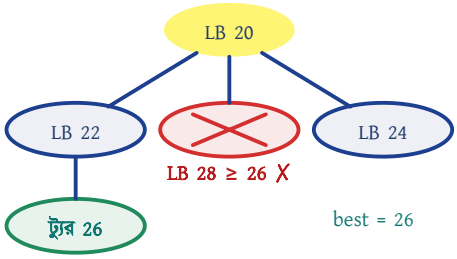
সাম অব সাবসেটস: S = {3, 5, 6, 7}, লক্ষ্য 15: প্রতিটি আইটেম নেওয়া/না-নেওয়া; যোগফল 15 ছাড়ালে বা আর পৌঁছাতে না পারলে ছাঁটাই। একমাত্র সমাধান **{3, 5, 7}**।

৬ ব্রাঞ্চ অ্যান্ড বাউন্ড: বাউন্ডে ট্রি ছাঁটা

সংজ্ঞা: **ব্রাঞ্চ অ্যান্ড বাউন্ড** (Branch and Bound, B&B)-ও স্টেট-স্পেস ট্রি খোঁজে, তবে **অপটিমাইজেশনের** জন্য। প্রতিটি জীবিত নোড পায় একটি **বাউন্ড** — ওই নোড দিয়ে সর্বোচ্চ কত ভালো উত্তর সম্ভব; বাউন্ড যদি **এ পর্যন্ত পাওয়া সেরা পূর্ণ সমাধানকে** হারাতে না পারে, নোডটি বাদ। ক্রম: FIFO (BFS-এর মতো), LIFO বা প্রায়োরিটি কিউ দিয়ে LC (least-cost) সার্চ। ব্যবহার: TSP (রিডিউসড কস্ট ম্যাট্রিক্স), 0/1 ন্যাপস্যাক (ভগ্নাংশ বাউন্ড), জব অ্যাসাইনমেন্ট।

পরীক্ষার ফাঁদ: ছাঁটাইয়ের নিয়ম: **মিনিমাইজেশনে** (TSP, অ্যাসাইনমেন্ট): LB(নোড) = যেকোনো পূর্ণতার ন্যূনতম খরচ; UB / best = এ পর্যন্ত পাওয়া সেরা পূর্ণ সমাধানের খরচ। ছাঁটাই করুন যখন **LB ≥ best** — LB সমান হলে বড়জোর সমান হয়, তাই সেটিও ছাঁটা হয় ("LB > UB" নিয়মে সমান-ক্ষেত্র বাদ পড়ে)। **ম্যাক্সিমাইজেশনে** (0/1 ন্যাপস্যাক): নোডের আপার বাউন্ড ≤ সেরা মান হলে ছাঁটাই।

মনে রাখার কৌশল: **"ব্যাকট্র্যাকিং জিজ্ঞেস করে 'এটা কি বৈধ?', B&B জিজ্ঞেস করে 'এটা কি এখনো জিততে পারে?'"** ব্যাকট্র্যাকিং = DFS + সম্ভাব্যতা; B&B = যেকোনো ক্রম (সাধারণত BFS / LC) + বাউন্ড।



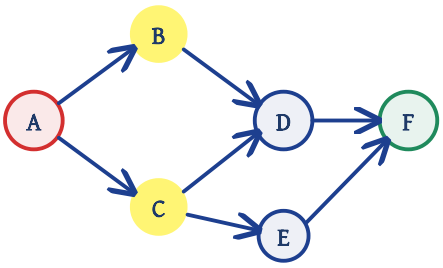
মিনিমাইজেশন: LB ≥ 26 হলেই বাদ

পদ্ধতি	মূল ভাবনা	উপসমস্যা	পরিচিত উদাহরণ
ডিভাইড অ্যান্ড কনকার	ভাগ, রিকার্শন, জোড়া	স্বাধীন	মার্জ সর্ট, বাইনারি সার্চ, স্ট্র্যাসেন
গ্রিডি	লোকাল সেরা, ফেরে না	একটিই বাছা	ভগ্নাংশ ন্যাপস্যাক, হাফম্যান, MST
ডায়নামিক প্রোগ্রামিং	উপ-উত্তর জমা	**ওভারল্যাপিং**	LCS, 0/1 ন্যাপস্যাক, MCM, ফ্লয়েড
ব্যাকট্র্যাকিং	DFS + সম্ভাব্যতা	স্টেট-স্পেস ট্রি	N-কুইন, সাবসেট সাম, কালারিং
ব্রাঞ্চ অ্যান্ড বাউন্ড	BFS / LC + বাউন্ড	স্টেট-স্পেস ট্রি	TSP, 0/1 ন্যাপস্যাক, অ্যাসাইনমেন্ট

৭ গ্রাফ ট্রাভার্সাল: BFS ও DFS

BFS: **ব্রেডথ-ফার্স্ট সার্চ** (Breadth-First Search) **কিউ** ব্যবহার করে: আগে উৎস, তারপর দূরত্ব ১-এর সব ভার্টেক্স, তারপর ২, ... এটি **ওজনহীন গ্রাফে শর্টেস্ট পাথ** (সবচেয়ে কম এজ) দেয়, বাইপার্টাইট যাচাই করে, এডমন্ডস-কার্প ও ওয়েব ক্রলার চালায়।

DFS: **ডেপথ-ফার্স্ট সার্চ** (Depth-First Search) **স্ট্যাক** (বা রিকার্শন) ব্যবহার করে: যত গভীরে যাওয়া যায় যাও, তারপর ফেরো। এটি ডিসকভারি/ফিনিশ সময় রাখে; সাইকেল শনাক্তকরণ, টপোলজিক্যাল সর্ট, SCC, আর্টিকুলেশন পয়েন্ট ও গোলকধাঁধা সমাধানের ভিত্তি।



A→B, A→C, B→D, C→D, C→E, D→F, E→F

	BFS	DFS
কাঠামো	কিউ (FIFO)	স্ট্যাক (LIFO) / রিকার্শন
A থেকে ক্রম (বর্ণানুক্রমে)	**A B C D E F**	**A B D F C E**
সময়	অ্যাডজেসেন্সি লিস্টে $O(V + E)$ · অ্যাডজেসেন্সি ম্যাট্রিক্সে $O(V^2)$	
ট্রি-তে যা দেয়	লেভেল-অর্ডার ট্রাভার্সাল	প্রি-, ইন-, পোস্ট-অর্ডার ট্রাভার্সাল

DFS এজ-শ্রেণি (ডিরেক্টেড): **ট্রি**: নতুন ভার্টেক্সে · **ব্যাক**: এখনো খোলা পূর্বপুরুষে · **ফরোয়ার্ড**: শেষ-হওয়া উত্তরসূরিতে · **ক্রস**: অন্য শেষ-হওয়া শাখার ভার্টেক্সে। আনডিরেক্টেড DFS-এ শুধু ট্রি ও ব্যাক এজ। **ব্যাক এজ আছে ⇔ গ্রাফে সাইকেল আছে।**

পরীক্ষার ফাঁদ: "DFS = প্রি/ইন/পোস্ট-অর্ডার, BFS = লেভেল অর্ডার" কথাটি শুধু **ট্রি**-র জন্য সত্য; সাধারণ গ্রাফে "ইন-অর্ডার" বলে কিছু নেই। BFS দেয় সবচেয়ে কম-**এজের** পাথ, সবচেয়ে কম-**ওজনের** নয়। বিচ্ছিন্ন গ্রাফে প্রতিটি অদেখা ভার্টেক্স থেকে নতুন BFS/DFS লাগে।

৮ টপোলজিক্যাল সর্টিং

সংজ্ঞা: ডিরেক্টেড গ্রাফের **টপোলজিক্যাল ক্রম** (topological order) ভার্টেক্সগুলোকে এমনভাবে সাজায় যে প্রতিটি এজ u → v-তে u আগে আসে। এমন ক্রম থাকে **যদি এবং কেবল যদি গ্রাফটি** DAG (ডিরেক্টেড অ্যাসাইক্লিক গ্রাফ) হয়, এবং সাধারণত তা **অনন্য নয়**। ব্যবহার: টাস্ক শিডিউলিং, কোর্সের পূর্বশর্ত, বিল্ড সিস্টেম (make), স্প্রেডশিট পুনর্গণনা, প্যাকেজ ইনস্টলার।

কানের অ্যালগরিদম (BFS) ✍️: ইন-ডিগ্রি A0 B1 C1 D2 E1 F2। ০-গুলো কিউতে: A। A বের → B, C হয় 0। B বের → D = 1। C বের → D = 0, E = 0। D বের → F = 1। E বের → F = 0। F বের। ক্রম **A B C D E F**। V-এর কম ভার্টেক্স বের হলে গ্রাফে সাইকেল আছে।

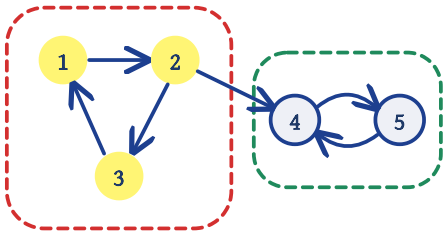
DFS পদ্ধতি ✍️: DFS চালান; কোনো ভার্টেক্স **শেষ** হলে তালিকায় রাখুন। শেষ হওয়ার ক্রম F, D, B, E, C, A → উল্টে: **A C E B D F**। কানের ক্রম থেকে আলাদা, কিন্তু সমান বৈধ। দুটিই $O(V + E)$।

মনে রাখার কৌশল: **"কান গোনে ইন-ডিগ্রি, DFS উল্টায় শেষের সময়।"** পূর্বশর্তের উদাহরণ: গণিত → ডেটা স্ট্রাকচার → অ্যালগরিদম, পদার্থবিজ্ঞান → রোবোটিক্স থেকে পাই গণিত, পদার্থবিজ্ঞান, ডেটা স্ট্রাকচার, অ্যালগরিদম, রোবোটিক্স।

৯ কানেক্টেড ও স্ট্রংলি কানেক্টেড কম্পোনেন্ট

আনডিরেক্টেড: **কানেক্টেড কম্পোনেন্ট** (connected component) হলো সর্বোচ্চ সংযুক্ত সাবগ্রাফ। প্রতিটি অদেখা ভার্টেক্স থেকে BFS/DFS শুরু করুন: শুরুর সংখ্যা = কম্পোনেন্টের সংখ্যা, $O(V + E)$।

ডিরেক্টেড: **স্ট্রংলি কানেক্টেড কম্পোনেন্ট** (SCC) হলো সর্বোচ্চ সেট যেখানে প্রতিটি ভার্টেক্স অন্য প্রতিটিতে পৌঁছায়। **কোসারাজু (Kosaraju):** (১) G-তে DFS, শেষ-ক্রম লিখুন; (২) ট্রান্সপোজ G^T বানান (সব এজ উল্টো); (৩) শেষ-সময়ের অধঃক্রমে G^T-তে DFS — প্রতিটি ট্রি একটি SCC। **টারজান (Tarjan):** স্ট্যাক ও low-link মান নিয়ে একটিই DFS; low[v] = disc[v] হলে v একটি SCC-র মূল। দুটিই **$O(V + E)$**।

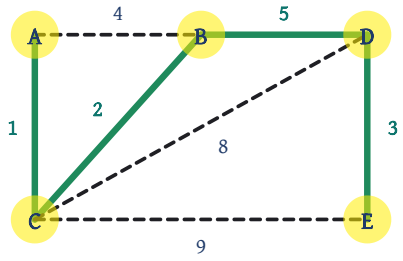


SCC: {1,2,3} ও {4,5}

পরীক্ষার ফাঁদ: প্রতিটি SCC-কে একটি নোডে সংকুচিত করলে পাই **কনডেনসেশন**, যা সবসময় DAG। **আর্টিকুলেশন পয়েন্ট** (cut vertex) এমন ভার্টেক্স যা সরালে গ্রাফ বিচ্ছিন্ন হয়; এমন এজকে বলে **ব্রিজ**। DFS low মান দিয়ে: মূল নোড কাট ভার্টেক্স যদি এর ≥ ২টি DFS সন্তান থাকে; অন্য ভার্টেক্স u কাট ভার্টেক্স যদি কোনো সন্তান v-এর low[v] ≥ disc[u]; এজ (u, v) ব্রিজ যদি low[v] > disc[u]।

১০ মিনিমাম স্প্যানিং ট্রি: প্রিম ও ক্রুসকাল

সংজ্ঞা: সংযুক্ত আনডিরেক্টেড গ্রাফের **স্প্যানিং ট্রি** (spanning tree)-তে সব V ভার্টেক্স ও ঠিক **V − 1টি এজ** থাকে, কোনো সাইকেল নেই। **মিনিমাম স্প্যানিং ট্রি** (MST)-র মোট ওজন সবচেয়ে কম। **কাট ধর্ম** (cut property): যেকোনো কাট পার হওয়া সবচেয়ে হালকা এজ কোনো না কোনো MST-তে থাকে — দুটি গ্রিডি অ্যালগরিদম সঠিক হওয়ার কারণ এটিই। সব ওজন ভিন্ন হলে MST অনন্য।



সবুজ = MST, ওজন 11

সমাধানকৃত উদাহরণ ✍️ একই গ্রাফ, দুই অ্যালগরিদম: এজ AB 4, AC 1, BC 2, BD 5, CD 8, DE 3, CE 9।

ক্রুসকাল (সাজাও, সাইকেল না হলে যোগ): AC 1 ✓, BC 2 ✓, DE 3 ✓, AB 4 ✗ (সাইকেল A-B-C), BD 5 ✓ → ৪টি এজ, থামুন।

প্রিম A থেকে (একটি ট্রি বাড়াও): AC 1 → CB 2 → BD 5 → DE 3।

দুটিতেই: **ওজন 1 + 2 + 3 + 5 = 11**, ৪টি = V − 1 এজ।

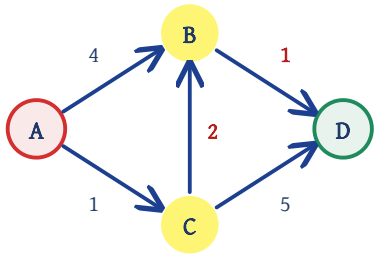
	প্রিম	ক্রুসকাল
কীভাবে	একটি ট্রি বাড়ায়; প্রান্তের ভার্টেক্সের প্রায়োরিটি কিউ	সাজানো এজ + **ইউনিয়ন-ফাইন্ড** ফরেস্ট
সময়	**$O(V^2)$** ম্যাট্রিক্স · $O(E \log V)$ বাইনারি হিপ · $O(E + V \log V)$ ফিবোনাচ্চি হিপ	$O(E \log E)$ = **$O(E \log V)$**
উপযুক্ত	**ঘন** (dense) গ্রাফ ($O(V^2)$ সংস্করণ)	**বিরল** (sparse) গ্রাফ

পরীক্ষার ফাঁদ: ঘন গ্রাফে ($E \approx V^2$) হিপ-প্রিমের খরচ $O(V^2 \log V)$, তাই সাধারণ **$O(V^2)$ ম্যাট্রিক্স প্রিম**-ই দ্রুততর। $E \le V^2$ বলে $\log E = O(\log V)$। MST ≠ শর্টেস্ট-পাথ ট্রি।

মনে রাখার কৌশল: **"প্রিম = পাশের এজ (ট্রি থেকে বাড়ে), ক্রুসকাল = কোথাও সবচেয়ে কম দামি এজ।"**

১১ শর্টেস্ট পাথ: স্থির না হওয়া পর্যন্ত রিল্যাক্স

রিল্যাক্সেশন: d[s] = 0, বাকি সব ∞ রেখে এজ **রিল্যাক্স (relax)** করুন: যদি d[u] + w(u, v) < d[v] তবে d[v] ← d[u] + w(u, v), parent[v] ← u। অ্যালগরিদমগুলোর পার্থক্য শুধু **কোন** এজ এবং **কোন ক্রমে** রিল্যাক্স করে তাতে।



ডিরেক্টেড, ওজন ছবির মতো

সমাধানকৃত উদাহরণ ✍️ A থেকে ডায়াক্সট্রা:

স্থির	A	B	C	D
শুরু	0	∞	∞	∞
A (0)	0	4	1	∞
C (1)	0	3	1	6
B (3)	0	3	1	4
D (4)	0	**3**	**1**	**4**

চূড়ান্ত: **A 0, C 1, B 3, D 4**; D-এর পাথ = A → C → B → D।

ডায়াক্সট্রা (গ্রিডি): বারবার সবচেয়ে ছোট d-ওয়ালা অমীমাংসিত ভার্টেক্সকে **স্থির** করে তার বহির্গামী এজ রিল্যাক্স করে; স্থির দূরত্বই চূড়ান্ত। শর্ত: **অঋণাত্মক ওজন**। সময়: $O(V^2)$ অ্যারে · $O((V+E) \log V)$ বাইনারি হিপ · $O(E + V \log V)$ ফিবোনাচ্চি হিপ।

পরীক্ষার ফাঁদ: ঋণাত্মক এজ: A→B 2, A→C 3, C→B −2। ডায়াক্সট্রা B-কে 2-তে স্থির করে আর ফিরে দেখে না, অথচ A→C→B = **1**। একটি ঋণাত্মক এজই একে ভাঙতে যথেষ্ট; বেলম্যান-ফোর্ড ব্যবহার করুন।

বেলম্যান-ফোর্ড (DP): **সব** এজ V − 1 বার রিল্যাক্স (শর্টেস্ট পাথে ≤ V − 1টি এজ)। আরও এক পাস: কোনো এজ তখনো রিল্যাক্স হলে উৎস থেকে পৌঁছানো যায় এমন **ঋণাত্মক-ওজনের সাইকেল** আছে। ঋণাত্মক এজ চলে। সময় **O(VE)**।

ফ্লয়েড-ওয়ারশাল (DP): সব জোড়া: k = 1..V, প্রতিটি i, j-এর জন্য d[i][j] = min(d[i][j], d[i][k] + d[k][j])। সময় **$O(V^3)$**, মেমরি $O(V^2)$। ঋণাত্মক এজ চলে; ঋণাত্মক সাইকেল দেখা যায় d[i][i] < 0 হিসেবে, তখন শর্টেস্ট পাথ অসংজ্ঞায়িত।

অ্যালগরিদম	পরিসর	পদ্ধতি	সময়	ঋণাত্মক এজ?
BFS	এক উৎস, ওজনহীন	কিউ	$O(V + E)$	প্রযোজ্য নয়
DAG শর্টেস্ট পাথ	এক উৎস, DAG	টপো ক্রম	$O(V + E)$	হ্যাঁ
ডায়াক্সট্রা	এক উৎস	গ্রিডি	$O((V+E) \log V)$	**না**
বেলম্যান-ফোর্ড	এক উৎস	DP	$O(VE)$	হ্যাঁ; ঋণাত্মক সাইকেল **ধরে**
ফ্লয়েড-ওয়ারশাল	সব জোড়া	DP	$O(V^3)$	হ্যাঁ; ঋণাত্মক সাইকেল **নয়**

পরীক্ষার ফাঁদ: বেলম্যান-ফোর্ড শুধু **ঋণাত্মক-ওজনের** সাইকেল ধরে (সাধারণ সাইকেল শনাক্ত করা DFS-এর কাজ)। ফ্লয়েড-ওয়ারশাল ঋণাত্মক সাইকেল দেখাতে পারে, কিন্তু তার পাশ কাটিয়ে উত্তর দিতে পারে না। ওয়ারশাল (একই লুপ, AND/OR) দেয় **ট্রানজিটিভ ক্লোজার**।

মনে রাখার কৌশল: "**ওজন নেই** → BFS, **ধনাত্মক** → **ডায়াক্সট্রা**, **ঋণাত্মক** → **বেলম্যান**, **সব জোড়া** → **ফ্লয়েড**।" জটিলতাও একই ক্রমে বাড়ে: V+E → E log V → VE → V^3।

পরীক্ষার আগের রাতের পাতা

সূত্র ও জটিলতা:

D&C: $T(n) = aT(n/b) + f(n)$

মার্জ সর্ট $2T(n/2)+n \to \Theta(n \log n)$

বাইনারি সার্চ $T(n/2)+1 \to \Theta(\log n)$

কুইক সর্ট খারাপতম $\Theta(n^2)$

স্ট্র্যাসেন $7T(n/2)+n^2 \to n^{2.81}$

কারাৎসুবা $3T(n/2)+n \to n^{1.585}$

LCS $O(mn)$ · এডিট ডিস্ট্যান্স $O(mn)$

0/1 ন্যাপস্যাক $O(nW)$ সিউডো-পলি

MCM $O(n^3)$, প্রতি গুণে খরচ pqr

ফিবোনাচ্চি DP $O(n)$, সাধারণ এক্সপো.

BFS/DFS লিস্টে $O(V+E)$, ম্যাট্রিক্সে $O(V^2)$

টপোলজিক্যাল সর্ট $O(V+E)$, শুধু DAG

SCC (কোসারাজু/টারজান) $O(V+E)$

MST: V − 1টি এজ

প্রিম $O(V^2)$ / $O(E \log V)$ / $O(E+V \log V)$

ক্রুসকাল $O(E \log E) = O(E \log V)$

ডায়াক্সট্রা $O((V+E) \log V)$, $w \geq 0$

বেলম্যান-ফোর্ড $O(VE)$

ফ্লয়েড-ওয়ারশাল $O(V^3)$, DP

DAG শর্টেস্ট পাথ $O(V+E)$

MCQ-র সেরা ১৪টি এক-লাইনার

১. D&C-র উপসমস্যা স্বাধীন; DP-র ওভারল্যাপিং

২. গ্রিডি: গ্রিডি-চয়েস + অপটিমাল সাবস্ট্রাকচার

৩. ভগ্নাংশ ন্যাপস্যাক → v/w অনুপাতে গ্রিডি

৪. 0/1 ন্যাপস্যাক → DP (গ্রিডি ব্যর্থ)

৫. হাফম্যান, প্রিম, ক্রুসকাল, ডায়াক্সট্রা = গ্রিডি

৬. ফ্লয়েড, বেলম্যান-ফোর্ড, LCS, MCM = DP

৭. ব্যাকট্র্যাকিং = স্টেট-স্পেস ট্রির DFS

৮. B&B: FIFO / LIFO / LC সার্চ

৯. BFS = কিউ, DFS = স্ট্যাক

১০. ব্যাক এজ ⇔ সাইকেল (ডিরেক্টেড DFS)

১১. টপোলজিক্যাল ক্রম আছে ⇔ DAG

১২. কোসারাজু: DFS, ট্রান্সপোজ, DFS

১৩. ৪-কুইন: ২টি সমাধান; ৮-কুইন: ৯২টি

১৪. বেলম্যান-ফোর্ড: V − 1 পাস + ১ যাচাই

প্রধান ফাঁদ: অনুপাত-গ্রিডি ভগ্নাংশ ন্যাপস্যাকে অপটিমাল, 0/1-এ নয় · কয়েন {1,3,4}-এ গ্রিডি ভাঙে · B&B মিনিমাইজেশনে LB ≥ best হলে ছাঁটাই · ১-D ন্যাপস্যাকে w নিচের দিকে চলে · "প্রি/ইন/পোস্ট বনাম লেভেল অর্ডার" শুধু ট্রি-র কথা · একটি ঋণাত্মক এজেই ডায়াক্সট্রা ব্যর্থ · বেলম্যান-ফোর্ড শুধু ঋণাত্মক-ওজনের সাইকেল ধরে · ফ্লয়েড-ওয়ারশাল ঋণাত্মক সাইকেল সামলাতে পারে না · ঘন গ্রাফ → $O(V^2)$ প্রিম · MST ≠ শর্টেস্ট-পাথ ট্রি · টপোলজিক্যাল ক্রম অনন্য নয়

সব কৌশল একসাথে: "ভাগ-জয়-জোড়া" · "আট নয়, সাত" (স্ট্র্যাসেন) · "গ্রিডি খপ করে ধরে, DP ভেবে দেখে" (সোনার গুঁড়া বনাম সোনার বার) · "মিললে → কোনাকুনি + 1" (LCS) · "ফ্লয়েড প্রতিটি k-এর ভেতর দিয়ে ভাসে" · "ব্যাকট্র্যাকিং: 'বৈধ কি?', B&B: 'এখনো জিততে পারে কি?'" · "কান গোনে ইন-ডিগ্রি, DFS উল্টায় শেষের সময়" · "প্রিম = পাশের এজ, ক্রুসকাল = কোথাও সবচেয়ে সস্তা এজ" · "ওজন নেই BFS, ধনাত্মক ডায়াক্সট্রা, ঋণাত্মক বেলম্যান, সব জোড়া ফ্লয়েড"।

✓ অনুশীলনী MCQ (পরীক্ষার ধরনে, ১০ নম্বর)

১. স্ট্র্যাসেনের ম্যাট্রিক্স গুণনের সময়:

ক) $O(n^3)$ খ) $O(n^{2.81})$

গ) $O(n^2 \log n)$ ঘ) $O(n^2)$

২. ফ্লয়েড-ওয়ারশাল অ্যালগরিদম ব্যবহার করে:

ক) গ্রিডি খ) ডিভাইড-কনকার

গ) ডায়নামিক প্রো. ঘ) ব্যাকট্র্যাকিং

৩. সর্বোচ্চ মান/ওজন অনুপাতে গ্রিডি সবসময় অপটিমাল ___ ন্যাপস্যাকে:

ক) 0/1 খ) ভগ্নাংশ
গ) বাউন্ডেড ঘ) বহু-শর্ত

৪. বেলম্যান-ফোর্ডের সময়-জটিলতা:

ক) $O(V + E)$ খ) $O(E \log V)$
গ) $O(V^3)$ ঘ) $O(VE)$

৫. উৎস থেকে পৌঁছানো যায় এমন ঋণাত্মক-ওজনের সাইকেল কোনটি ধরে?

ক) ডায়াক্সট্রা খ) প্রিম
গ) বেলম্যান-ফোর্ড ঘ) ক্রুসকাল

৬. টপোলজিক্যাল ক্রম থাকে যদি এবং কেবল যদি গ্রাফটি:

ক) DAG খ) কমপ্লিট
গ) বাইপার্টাইট ঘ) সাইক্লিক

৭. $T(n) = 2T(n/2) + n$-এর সমাধান:

ক) $\Theta(n)$ খ) $\Theta(n \log n)$
গ) $\Theta(n^2)$ ঘ) $\Theta(\log n)$

৮. এজ A→B 4, A→C 1, C→B 2, B→D 1, C→D 5। A থেকে D-এর শর্টেস্ট দূরত্ব:

ক) ৬ খ) ৫
গ) ৪ ঘ) ৩

৯. ১০টি ভার্টেক্সের সংযুক্ত গ্রাফের MST-তে এজ কয়টি?

ক) ১০ খ) ৪৫
গ) ১১ ঘ) ৯

১০. মিনিমাইজেশন সমস্যায় ব্রাঞ্চ অ্যান্ড বাউন্ডে নোড ছাঁটা হয় যখন:

ক) LB ≥ best খ) LB < best
গ) এটি পাতা ঘ) এটি মূল

উত্তরমালা → ১-খ · ২-গ · ৩-খ · ৪-ঘ · ৫-গ · ৬-ক · ৭-খ · ৮-গ · ৯-ঘ · ১০-ক

লিখিত প্রশ্ন: চারটি ম্যাট্রিক্স $A_1 \dots A_4$-এর মাপ $p = (5, 4, 6, 2, 7)$। ম্যাট্রিক্স-চেইন গুণনের ডায়নামিক প্রোগ্রামিং রিকারেন্স লিখুন, খরচের টেবিল পূরণ করুন, অপটিমাল বন্ধনী-বিন্যাস দিন এবং সময়-জটিলতা বলুন। ১০

১ **রিকারেন্স** — $m[i][i] = 0$; $m[i][j] = \min_{i \le k < j} \{ m[i][k] + m[k+1][j] + p_{i-1} p_k p_j \}$। ২

২ **২-এর চেইন** — $m[1][2] = 5 \cdot 4 \cdot 6 = 120$; $m[2][3] = 4 \cdot 6 \cdot 2 = 48$; $m[3][4] = 6 \cdot 2 \cdot 7 = 84$। ২

৩ **৩-এর চেইন** — $m[1][3] = \min(0 + 48 + 5 \cdot 4 \cdot 2 = 88,\ 120 + 0 + 5 \cdot 6 \cdot 2 = 180) = 88$ ($k = 1$); $m[2][4] = \min(0 + 84 + 4 \cdot 6 \cdot 7 = 252,\ 48 + 0 + 4 \cdot 2 \cdot 7 = 104) = 104$ ($k = 3$)। ২

৪ **পূর্ণ চেইন** — $m[1][4] = \min(k{=}1{:}\ 0 + 104 + 140 = 244;\ k{=}2{:}\ 120 + 84 + 210 = 414;\ k{=}3{:}\ 88 + 0 + 70 = 158) =$ **158**, বিভাজন $k = 3$। ২

৫ **উত্তর** — $((A_1(A_2A_3))A_4)$, ১৫৮টি গুণ; সময় $O(n^3)$, মেমরি $O(n^2)$। ২

পরীক্ষার আগে ✓: ☐ মার্জ সর্ট, বাইনারি সার্চ, স্ট্র্যাসেনের রিকারেন্স · ☐ ভগ্নাংশ বনাম 0/1 ন্যাপস্যাক একবার কষে দেখা · ☐ LCS ও ন্যাপস্যাক টেবিল হাতে · ☐ ৪-কুইনের ব্যাকট্র্যাক · ☐ দুই দিকেই B&B ছাঁটাইয়ের নিয়ম · ☐ BFS/DFS ক্রম ও এজ-শ্রেণি · ☐ ছোট DAG-এ কানের অ্যালগরিদম · ☐ এক গ্রাফে ক্রুসকাল ও প্রিম · ☐ ডায়াক্সট্রার টেবিল · ☐ শর্টেস্ট-পাথ তুলনার টেবিল